

ASSIGNMENT-11.2

H.no-2303A51927

Batch – 30

Task 1 – Stack Using AI Guidance

Prompt :

Design and implement a Stack data structure supporting basic stack operations. A Python Stack class supporting push, pop, peek, and empty- check operations with proper documentation.

CODE:

```
AI-11.2.py > ...
1  #design and implement a Stack data structure supporting basic stack operations.A Python Stack class supporting push, pop, peek
2  class Stack:
3      def __init__(self):
4          """Initialize an empty stack."""
5          self.stack = []
6
7      def push(self, item):
8          """Add an item to the top of the stack.
9
10         Args:
11             item: The item to be added to the stack.
12         """
13         self.stack.append(item)
14
15      def pop(self):
16          """Remove and return the item at the top of the stack.
17
18         Returns:
19             The item at the top of the stack.
20
21         Raises:
22             IndexError: If the stack is empty.
23         """
24         if self.empty():
25             raise IndexError("Pop from an empty stack")
26         return self.stack.pop()
27
28      def peek(self):
29          """Return the item at the top of the stack without removing it.
30
31         Returns:
32             The item at the top of the stack.
33
34         Raises:
35             IndexError: If the stack is empty.
36         """
37         if self.empty():
38             raise IndexError("Peek from an empty stack")
39         return self.stack[-1]
40
41      def empty(self):
42          """Check if the stack is empty.
43
44         Returns:
45             True if the stack is empty, False otherwise.
46         """
47         return len(self.stack) == 0
```

```

37     if self.empty():
38         raise IndexError("Peek from an empty stack")
39     return self.stack[-1]
40
41     def empty(self):
42         """Check if the stack is empty.
43
44         Returns:
45             True if the stack is empty, False otherwise.
46         """
47         return len(self.stack) == 0
48     # Example usage:
49     if __name__ == "__main__":
50         stack = Stack()
51         stack.push(1)
52         stack.push(2)
53         stack.push(3)
54         print(stack.peak()) # Output: 3
55         print(stack.pop()) # Output: 3
56         print(stack.peak()) # Output: 2
57         print(stack.empty()) # Output: False
58         stack.pop()
59         stack.pop()
60         print(stack.empty()) # Output: True
61

```

OUTPUT:

```

3
3
2
False
True
PS D:\AI>

```

JUSTIFICATION:

A stack is an important data structure that follows the Last In First Out (LIFO) principle. It is widely used in applications such as function calls, recursion, and undo/redo operations in editors. Stacks also help in expression evaluation and syntax parsing in compilers.

Task 2 – Queue Design

Prompt :

create a Queue data structure following FIFO principles. A complete Queue implementation including enqueue, dequeue, front element access, and size calculation

CODE:

```

66 #create a Queue data structure following FIFO principles.A complete Queue implementation including enqueue, dequeue,front element acc
67 class Queue:
68     def __init__(self):
69         """Initialize an empty queue."""
70         self.queue = []
71
72     def enqueue(self, item):
73         """Add an item to the end of the queue.
74
75         Args:
76             item: The item to be added to the queue.
77         """
78         self.queue.append(item)
79
80     def dequeue(self):
81         """Remove and return the item at the front of the queue.
82
83         Returns:
84             The item at the front of the queue.
85
86         Raises:
87             IndexError: If the queue is empty.
88         """
89         if self.empty():
90             raise IndexError("Dequeue from an empty queue")
91         return self.queue.pop(0)
92
93     def front(self):
94         """Return the item at the front of the queue without removing it.
95
96         Returns:
97             The item at the front of the queue.
98
99         Raises:
100             IndexError: If the queue is empty.
101         """
102         if self.empty():
103             raise IndexError("Front from an empty queue")
104         return self.queue[0]
105
106     def size(self):
107         """Return the number of items in the queue.
108
109         Returns:
110             The size of the queue.
111         """
112         return len(self.queue)

```

```

102     if self.empty():
103         raise IndexError("Front from an empty queue")
104     return self.queue[0]
105
106     def size(self):
107         """Return the number of items in the queue.
108
109         Returns:
110             The size of the queue.
111         """
112         return len(self.queue)
113
114     def empty(self):
115         """Check if the queue is empty.
116
117         Returns:
118             True if the queue is empty, False otherwise.
119         """
120         return len(self.queue) == 0
121
122     # Example usage:
123     if __name__ == "__main__":
124         queue = Queue()
125         queue.enqueue(1)
126         queue.enqueue(2)
127         queue.enqueue(3)
128
129         print(queue.front()) # Output: 1
130         print(queue.dequeue()) # Output: 1
131         print(queue.front()) # Output: 2
132         print(queue.size()) # Output: 2
133         queue.dequeue()
134         queue.dequeue()
135         print(queue.empty()) # Output: True
136

```

OUTPUT:

```
1
1
2
2
True
PS D:\AI>
```

JUSTIFICATION:

A queue follows the First In First Out (FIFO) principle, ensuring elements are processed in the order they arrive. It is commonly used in CPU scheduling, printer task management, and network data handling. Queues help manage tasks efficiently in systems where order is important.

Task 3 – Singly Linked List Construction

Prompt :

build a singly linked list supporting insertion and traversal. Correctly functioning linked list with node creation, insertion logic, and display functionality.

CODE:

```
138 #build a singly linked list supporting insertion and traversal.Correctly functioning linked list with node creation, insertion logic, and display functionality
139 class Node:
140     def __init__(self, data):
141         """Initialize a node with data and a pointer to the next node."""
142         self.data = data
143         self.next = None
144 class SinglyLinkedList:
145     def __init__(self):
146         """Initialize an empty singly linked list."""
147         self.head = None
148
149     def insert(self, data):
150         """Insert a new node with the given data at the end of the list.
151
152         Args:
153             data: The data to be stored in the new node.
154         """
155         new_node = Node(data)
156         if self.head is None:
157             self.head = new_node
158             return
159         last_node = self.head
160         while last_node.next:
161             last_node = last_node.next
162         last_node.next = new_node
163
164     def display(self):
165         """Display the contents of the linked list."""
166         current_node = self.head
167         while current_node:
168             print(current_node.data, end=' ')
169             current_node = current_node.next
170         print() # for a new line after displaying the list
171
172 # Example usage:
173 if __name__ == "__main__":
174     linked_list = SinglyLinkedList()
175     linked_list.insert(1)
176     linked_list.insert(2)
177     linked_list.insert(3)
178     linked_list.display() # Output: 1 2 3
```

OUTPUT:

```
1 2 3
PS D:\AI>
```

JUSTIFICATION:

A singly linked list is a dynamic data structure that allows efficient insertion and deletion of elements. It is useful when the size of data changes frequently. Linked lists are also used in implementing stacks, queues, and memory management systems.

Task 4 – Binary Search Tree Operations

Prompt :

Implement a Binary Search Tree support focusing on insertion and traversal. BST program with correct node insertion and in-order traversal output.

CODE:

```
182 #Implement a Binary Search Tree with AI support focusing on insertion and traversal.BST program with correct node insertion and in-order traversal output.
183 class TreeNode:
184     def __init__(self, key):
185         """Initialize a tree node with a key and pointers to left and right children."""
186         self.left = None
187         self.right = None
188         self.val = key
189 class BinarySearchTree:
190     def __init__(self):
191         """Initialize an empty binary search tree."""
192         self.root = None
193     def insert(self, key):
194         """Insert a new key into the binary search tree.
195
196         Args:
197             key: The key to be inserted into the BST.
198         """
199         if self.root is None:
200             self.root = TreeNode(key)
201         else:
202             self._insert_recursively(self.root, key)
203     def _insert_recursively(self, node, key):
204         """Helper method to insert a key recursively."""
205         if key < node.val:
206             if node.left is None:
207                 node.left = TreeNode(key)
208             else:
209                 self._insert_recursively(node.left, key)
210         else:
211             if node.right is None:
212                 node.right = TreeNode(key)
213             else:
214                 self._insert_recursively(node.right, key)
215     def in_order_traversal(self):
216         """Perform in-order traversal of the BST and return a list of keys."""
217         return self._in_order_recursively(self.root)
218     def _in_order_recursively(self, node):
219         """Helper method for in-order traversal recursively."""
220         res = []
221         if node:
222             res = self._in_order_recursively(node.left)
223             res.append(node.val)
224             res = self._in_order_recursively(node.right)
```

```

213         else:
214             if node.right is None:
215                 node.right = TreeNode(key)
216             else:
217                 self._insert_recursively(node.right, key)
218
219         def in_order_traversal(self):
220             """Perform in-order traversal of the BST and return a list of keys."""
221             return self._in_order_recursively(self.root)
222
223         def _in_order_recursively(self, node):
224             """Helper method for in-order traversal recursively."""
225             res = []
226             if node:
227                 res = self._in_order_recursively(node.left)
228                 res.append(node.val)
229                 res = res + self._in_order_recursively(node.right)
230             return res
231
232         # Example usage:
233         if __name__ == "__main__":
234             bst = BinarySearchTree()
235             bst.insert(5)
236             bst.insert(3)
237             bst.insert(7)
238             bst.insert(2)
239             bst.insert(4)
240
241             print(bst.in_order_traversal()) # Output: [2, 3, 4, 5, 7]
242

```

OUTPUT:

```

[2, 3, 4, 5, 7]
PS D:\AI>

```

JUSTIFICATION:

A Binary Search Tree (BST) organizes data in a sorted hierarchical structure, allowing faster searching and insertion operations. It improves efficiency compared to linear search. BSTs are widely used in database indexing, searching algorithms, and file systems.

Task 5 – Hash Table Implementation

Prompt :

Create a hash table with collision handling. Hash table supporting insert, search, and delete using chaining or open.

CODE:

```

245 #Create a hash table using AI with collision handling.Hash table supporting insert, search, and delete using chaining or open.
246 class HashTable:
247     def __init__(self, size=10):
248         """Initialize a hash table with a specified size."""
249         self.size = size
250         self.table = [[] for _ in range(size)]
251
252     def _hash(self, key):
253         """Generate a hash for the given key."""
254         return hash(key) % self.size
255
256     def insert(self, key, value):
257         """Insert a key-value pair into the hash table.
258
259         Args:
260             key: The key to be inserted.
261             value: The value associated with the key.
262         """
263         index = self._hash(key)
264         for i, (k, v) in enumerate(self.table[index]):
265             if k == key:
266                 self.table[index][i] = (key, value) # Update existing key
267                 return
268         self.table[index].append((key, value)) # Insert new key-value pair
269
270     def search(self, key):
271         """Search for a value by its key in the hash table.
272
273         Args:
274             key: The key to search for.
275
276         Returns:
277             The value associated with the key if found, else None.
278         """
279         index = self._hash(key)
280         for k, v in self.table[index]:
281             if k == key:
282                 return v
283         return None
284
285     def delete(self, key):
286         """Delete a key-value pair from the hash table by its key.
287
288         Args:
289             key: The key to be deleted.
290

```

```

285     def delete(self, key):
286         """Delete a key-value pair from the hash table by its key.
287
288         Args:
289             key: The key to be deleted.
290
291         Returns:
292             True if the key was found and deleted, False otherwise.
293         """
294         index = self._hash(key)
295         for i, (k, v) in enumerate(self.table[index]):
296             if k == key:
297                 del self.table[index][i]
298                 return True
299         return False
300 # Example usage:
301 if __name__ == "__main__":
302     hash_table = HashTable()
303     hash_table.insert("name", "Alice")
304     hash_table.insert("age", 30)
305     hash_table.insert("city", "New York")
306
307     print(hash_table.search("name")) # Output: Alice
308     print(hash_table.search("age")) # Output: 30
309     print(hash_table.search("city")) # Output: New York
310
311     hash_table.delete("age")
312     print(hash_table.search("age")) # Output: None
313
314

```

OUTPUT:

```
Alice  
30  
New York  
None  
PS D:\AI>
```

JUSTIFICATION:

A hash table is used for fast data storage and retrieval using key-value mapping. It provides very efficient search operations with average constant time complexity $O(1)$. Hash tables are widely used in databases, caches, dictionaries, and indexing systems.